

ASR Assignment Report

Hindi Automatic Speech Recognition Pipeline

AI Researcher Intern | Speech & Audio | Josh Talks

Submitted By	Daksh
Date	March 2026
Tools Used	Python, HuggingFace Transformers, PyTorch, Kaggle T4 GPU
Role	AI Researcher Intern - Speech & Audio

Overview

This assignment involved building and evaluating a Hindi Automatic Speech Recognition (ASR) pipeline end-to-end from raw audio data to fine-tuned model evaluation, text cleaning, spelling correction, and advanced evaluation methodology. Each question tackled a real production problem that ASR teams face daily.

Question 1 : Whisper Fine-tuning on Hindi ASR Data

The Problem

We were given approximately **10 hours** of Hindi conversational audio recorded by real speakers across India. The goal was to fine-tune OpenAI's **Whisper-small** model on this data and evaluate improvement over the baseline.

Data Challenges

Challenge 1 : Broken URLs

The dataset sheet pointed to a GCP bucket that returned **404 errors** for all files. We discovered the correct base URL by testing multiple URL patterns programmatically and found that replacing `joshtalks-data-collection/hq_data/hi/` with `upload_goai/` resolved all 104 recordings successfully.

Challenge 2 : Dataset Quality

After downloading all transcription JSONs, the raw data had significant noise. Out of **5,941 total segments**, we removed 1,498:

- **209** segments had REDACTED labels
- **1,012** segments were under 1 second (too short for meaningful speech)
- **878** segments had fewer than 5 characters of text

This left **4,442 clean segments**.

Challenge 3 : Audio Format

All 104 audio files were at **44,100 Hz**. Whisper requires **16,000 Hz**. We resampled all files using **librosa** before slicing them into individual segments using ground-truth timestamps.

Preprocessing Pipeline Summary

Step	Input	Output
URL fixing	104 broken URLs	104 working URLs

Transcription download	104 JSON files	5,941 segments
Quality filtering	5,941 segments	4,442 clean segments
Audio resampling	44,100 Hz WAV	16,000 Hz WAV
Segment slicing	104 full recordings	4,442 audio clips
Train/val split	4,442 segments	4,093 train / 349 val

Final dataset: 4,442 segments, 11.44 hours, 102 unique speakers

Fine-tuning Setup

We fine-tuned **Whisper-small (241.7M parameters)** using the HuggingFace **Seq2SeqTrainer** with the following key settings:

Hyperparameter	Value
Learning Rate	1e-5
Effective Batch Size	32 (batch 4 × gradient accumulation 8)
Epochs	3
Precision	FP16 (half precision for GPU efficiency)
Gradient Checkpointing	Enabled (to fit in 14.6 GB GPU memory)

Training Challenges Encountered

- **OutOfMemoryError:** Batch size 16 caused GPU OOM. Reduced to batch size 4 with gradient accumulation 8 to maintain effective batch size of 32.
- **Transformers v5 Compatibility Bug:** Library no longer allows setting generation parameters via `model.config`. Fix: set all generation parameters exclusively through `model.generation_config`.

Results

Model	Dataset	WER
Whisper-small (Baseline)	Hindi val set	1.2537
Whisper-small (Fine-tuned)	Hindi val set	0.4028
Improvement	—	↓ 67.8%

Training Loss Progression Across 3 Epochs

Epoch	Training Loss	Validation Loss	WER
1	13.22	0.657	0.546
2	6.98	0.471	0.435
3	5.07	0.414	0.403

Error Analysis

Sampling Strategy

Inference was run on **100 validation samples**; **91 with errors** were identified. These were sorted by WER descending and sampled at every 3rd index (stride sampling) to get **25 representative samples** covering WER=4.9 down to WER=0.27. This avoids cherry-picking and gives a fair severity distribution.

Error Taxonomy (25 Sampled Errors)

Category	Count	%	Example
Phonetic Confusion	10	40%	महान खिलाड़ी → मान के अड़ी
Spelling Variation	7	28%	इधर → इदर, वगैरह → वगैरा
English Loanword Error	4	16%	अफॉर्डेबल → पॉइ बोल
Filler Word Confusion	3	12%	हु हु → हूं हूं
Hallucination / Repetition	1	4%	Noisy audio → आ आ आ... (100x)

Root Causes

Phonetic Confusion: The dominant error type. Whisper's tokenizer was trained on formal, clean Hindi text. Conversational Hindi from rural speakers on cheap mobile devices features fast speech, reduced vowels, and regional accents the model has not fully adapted to even after fine-tuning.

Spelling Variation: Partially unfair to the model, Hindi has no single standardized orthography. Words like वगैरह/वगैरा and इधर/इदर are both valid spellings. Standard WER penalizes these unnecessarily.

English Loanword Errors: English words like अफॉर्डेबल and यूनिवर्सिटी are pronounced differently by every speaker. Without a consistent phoneme-to-grapheme mapping for loanwords, the model guesses incorrectly.

Proposed Fixes

- **Repetition Loop Detection** : Post-process outputs to collapse tokens repeated 4+ times. Addresses hallucination on noisy audio.
- **Spelling Normalization Dictionary** : Map common dialectal variants to standard spellings post-inference. Reduces unfair WER penalty.
- **English Loanword Lexicon** : Build a constrained decoding list of valid Devanagari transliterations for common English words. Forces model to output a known valid form.

Before / After Results (Fixes 1 & 2 Implemented)

Metric	Before	After	Change
Overall WER (25 samples)	0.5867	0.4241	↓ 27.7%
Sample 1 (repetition)	4.913	1.000	↓ 79.6%
Sample 16 (spelling)	0.375	0.208	↓ 44.5%
Sample 19 (spelling)	0.342	0.211	↓ 38.3%

Question 2 : Text Cleaning Pipeline

The Problem

Raw ASR output is messy. Numbers come out as words, and English words spoken in Hindi conversations appear inconsistently. We built a two-stage cleaning pipeline to handle both.

Part A : Number Normalization

We built a **rule-based converter** that handles three levels of complexity:

- **Simple:** दो → 2, दस → 10, सौ → 100
- **Compound:** तीन सौ चौवन → 354, एक हज़ार → 1000, एक लाख → 100000
- **Edge cases with judgment calls**

Input	Output	Decision
दो-चार बातें	दो-चार बातें	Idiom — hyphenated, kept as-is
दो चार लोग थे	दो चार लोग थे	Idiom — colloquial expression, not numeric
एक नदी	1 नदी	Known limitation — एक acts as article here
एक हज़ार रुपये दो	1000 रुपये 2	दो is ambiguous — numeric context dominates

The **idiom protection** works by replacing known idiom phrases with placeholders before number conversion, then restoring them after.

Part B : English Word Detection

We built a **dictionary-based tagger** with 80+ common English loanwords written in Devanagari, plus Roman script detection. Output uses [EN] . . . [/EN] tags.

Example Input: मेरा इंटरव्यू बहुत अच्छा गया

Example Output: मेरा [EN]इंटरव्यू[/EN] बहुत अच्छा गया

Ambiguous Words Deliberately Excluded

Word	Reason for Exclusion
बिल	Hindi (hole) and English (bill) — ambiguous
बस	Primary meaning is Hindi (enough)
कार	Fully absorbed into Hindi lexicon

Key Tradeoff: A dictionary-based approach has high precision but limited recall. Words not in the dictionary are missed. A better long-term solution would be a trained classifier using character n-gram features to distinguish Hindi from Devanagari-transliterated English.

Question 3 : Spelling Correction at Scale

The Problem

Given **1,77,508 unique words** from human transcriptions, classify each as correctly or incorrectly spelled, with a confidence score.

Our Approach : 3-Layer Classifier

Layer 1 : Hard Rules (High Confidence)

- **Roman script** → Incorrect
- **Starts with matra character** → Incorrect (phonologically impossible in Hindi)
- **Punctuation attached to word** → Incorrect
- **Found in dictionary** → Correct

Layer 2 : Morphological Rules (Medium Confidence)

- **Valid Hindi suffix** (ना, ता, ती, वाला, वाली etc.) → Correct
- **Suspicious repetition pattern** → Incorrect
- **Likely merged words** → Incorrect

Layer 3 : Devanagari Validity (Low Confidence)

- **Valid Devanagari characters but not in dictionary** → Correct (uncertain)

Classification Results

Category	Count
Total Words	1,77,508
Correctly Spelled	1,48,077
Incorrectly Spelled	29,431

Confidence Breakdown

Confidence Level	Correct	Incorrect
High	7,184	28,516
Medium	20,147	905
Low	1,20,746	10

Low Confidence Analysis

Manual review of **45 low-confidence words** showed system accuracy of **82.5%** on clear cases. The main failure mode: the system labels everything as 'correct' if it passes the Devanagari validity check, even when clearly wrong.

Two Categories Where System Is Unreliable

- **Merged Words** : e.g., सहीहै, वहीहै, साफसुथरा are two words joined without a space. They pass all rules because they are valid Devanagari sequences. Fixing this requires a word segmentation model.
- **Dialectal Spellings** : e.g., वगरह (वगैरह), अंखों (आँखों) are phonetically valid but orthographically non-standard. A rule-based system cannot distinguish genuine errors from valid dialectal variants without a pronunciation lexicon.

Question 4 : Lattice-Based WER Evaluation

Why Standard WER is Flawed

Standard WER compares model output against one rigid reference string. This unfairly penalizes models that produce valid alternative transcriptions:

- **Reference:** उसने चौदह किताबें खरीदीं
- **Model Output 1:** उसने 14 किताबें खरीदीं → WER = 25% (**unfair - numerically equivalent**)
- **Model Output 2:** उसने चौदह पुस्तकें खरीदीं → WER = 25% (**unfair - semantically equivalent**)

Both outputs are semantically correct but get penalized by standard WER.

Lattice Design

A lattice replaces the flat reference string with a sequential list of **bins**. Each bin contains all valid alternatives at that position, enabling fairer evaluation:

Position	Bin – Valid Alternatives
1	उसने
2	चौदह 14
3	किताबें पुस्तकें
4	खरीदीं खरीदी

This approach gives **fairer evaluation** by accepting all valid alternatives at each position, reducing the unfair WER penalty for semantically and numerically equivalent transcriptions.

Summary of Achievements

Question	Task	Key Result
Q1 Fine-tuning	Whisper-small on Hindi ASR data	67.8% WER reduction (1.25 → 0.40)
Q1 Post-processing	Repetition & spelling fixes	Additional 27.7% WER reduction
Q2A	Hindi word-to-digit conversion	Rule-based with idiom protection
Q2B	Devanagari loanword tagging	Dictionary tagger, 80+ words covered
Q3	3-layer rule-based spelling classifier	82.5% accuracy on low-confidence set
Q4	Fair evaluation framework	Multi-alternative bin-based lattice WER

Kaggle Notebook link : <https://www.kaggle.com/code/daksh159/josh-talks-q1-preprocessing>

Final CSV with 1,77,508 words with correct/incorrect labels : [q3_final_utf8.csv](#)